Practice Session on Dijkstra's Algorithm

Understanding Dijkstra's Algorithm

What is Dijkstra's Algorithm?

Dijkstra's algorithm is a greedy algorithm used to find the shortest path from a source vertex to all other vertices in a weighted graph with non-negative edge weights. It works by maintaining a set of vertices whose shortest distance from the source is known and gradually expanding this set.

Key Concepts

- **Relaxation**: The process of updating the distance to a vertex if a shorter path is found
- **Greedy Choice**: Always select the unvisited vertex with the minimum distance
- **Optimal Substructure**: The shortest path to any vertex consists of shortest paths to intermediate vertices

Algorithm Steps

1. Initialize distances of all vertices to infinity, except the source vertex (distance = 0)
2. Create a priority queue and insert the source vertex
3. While the priority queue is not empty:
    - Extract the vertex with minimum distance
    - For each adjacent vertex, check if we can relax the edge
    - If relaxation is possible, update the distance and add to priority queue
4. Repeat until all vertices are processed

Basic Implementation

Graph Representation

```java
import java.util.*;

class Edge {
    int destination;
    int weight;

    public Edge(int dest, int weight) {
        this.destination = dest;
        this.weight = weight;
    }
}

class Graph {
    private int vertices;
    private List<List<Edge>> adjacencyList;

    public Graph(int vertices) {
        this.vertices = vertices;
        this.adjacencyList = new ArrayList<>();

        for (int i = 0; i < vertices; i++) {
            adjacencyList.add(new ArrayList<>());
        }
    }

    public void addEdge(int source, int destination, int weight) {
        adjacencyList.get(source).add(new Edge(destination, weight));
    }

    public List<List<Edge>> getAdjacencyList() {
        return adjacencyList;
    }

    public int getVertices() {
        return vertices;
    }
}
```

Node Class for Priority Queue

```java
class Node implements Comparable<Node> {
    int vertex;
    int distance;

    public Node(int vertex, int distance) {
        this.vertex = vertex;
        this.distance = distance;
    }

    @Override
    public int compareTo(Node other) {
        return Integer.compare(this.distance, other.distance);
    }
}
```

Standard Dijkstra's Implementation

Finding Shortest Distances

java

```java
public class DijkstraBasic {

    public static int[] dijkstra(Graph graph, int source) {
        int vertices = graph.getVertices();
        int[] distance = new int[vertices];
        boolean[] visited = new boolean[vertices];

        // Initialize distances
        Arrays.fill(distance, Integer.MAX_VALUE);
        distance[source] = 0;

        PriorityQueue<Node> pq = new PriorityQueue<>();
        pq.offer(new Node(source, 0));

        while (!pq.isEmpty()) {
            Node current = pq.poll();
            int u = current.vertex;

            if (visited[u]) continue;
            visited[u] = true;

            // Check all adjacent vertices
            for (Edge edge : graph.getAdjacencyList().get(u)) {
                int v = edge.destination;
                int weight = edge.weight;

                // Relaxation step
                if (!visited[v] && distance[u] != Integer.MAX_VALUE &&
                    distance[u] + weight < distance[v]) {
                    distance[v] = distance[u] + weight;
                    pq.offer(new Node(v, distance[v]));
                }
            }
        }

        return distance;
    }

    public static void printDistances(int[] distances, int source) {
        System.out.println("Shortest distances from vertex " + source + ":");
        for (int i = 0; i < distances.length; i++) {
            if (distances[i] == Integer.MAX_VALUE) {
                System.out.println("Vertex " + i + ": INF");
            } else {
                System.out.println("Vertex " + i + ": " + distances[i]);
            }
```

```
            }
        }
    }
```

Advanced Implementation with Path Tracking

Dijkstra with Path Reconstruction

java

```java
public class DijkstraWithPath {

    static class Result {
        int[] distances;
        int[] previous;

        public Result(int[] distances, int[] previous) {
            this.distances = distances;
            this.previous = previous;
        }
    }

    public static Result dijkstraWithPath(Graph graph, int source) {
        int vertices = graph.getVertices();
        int[] distance = new int[vertices];
        int[] previous = new int[vertices];
        boolean[] visited = new boolean[vertices];

        Arrays.fill(distance, Integer.MAX_VALUE);
        Arrays.fill(previous, -1);
        distance[source] = 0;

        PriorityQueue<Node> pq = new PriorityQueue<>();
        pq.offer(new Node(source, 0));

        while (!pq.isEmpty()) {
            Node current = pq.poll();
            int u = current.vertex;

            if (visited[u]) continue;
            visited[u] = true;

            for (Edge edge : graph.getAdjacencyList().get(u)) {
                int v = edge.destination;
                int weight = edge.weight;

                if (!visited[v] && distance[u] != Integer.MAX_VALUE &&
                    distance[u] + weight < distance[v]) {
                    distance[v] = distance[u] + weight;
                    previous[v] = u;
                    pq.offer(new Node(v, distance[v]));
                }
            }
        }

        return new Result(distance, previous);
```

```java
    }

    public static List<Integer> getPath(int[] previous, int source, int destination) {
        List<Integer> path = new ArrayList<>();

        if (previous[destination] == -1 && source != destination) {
            return path; // No path exists
        }

        int current = destination;
        while (current != -1) {
            path.add(current);
            current = previous[current];
        }

        Collections.reverse(path);
        return path;
    }

    public static void printPath(List<Integer> path, int source, int destination) {
        if (path.isEmpty()) {
            System.out.println("No path from " + source + " to " + destination);
        } else {
            System.out.print("Path from " + source + " to " + destination + ": ");
            for (int i = 0; i < path.size(); i++) {
                System.out.print(path.get(i));
                if (i < path.size() - 1) {
                    System.out.print(" -> ");
                }
            }
            System.out.println();
        }
    }
}
```

Optimized Implementation Using Arrays

Array-Based Priority Queue

```java
public class DijkstraOptimized {

    public static int[] dijkstraArray(Graph graph, int source) {
        int vertices = graph.getVertices();
        int[] distance = new int[vertices];
        boolean[] visited = new boolean[vertices];

        Arrays.fill(distance, Integer.MAX_VALUE);
        distance[source] = 0;

        for (int count = 0; count < vertices - 1; count++) {
            int u = minDistance(distance, visited);
            visited[u] = true;

            for (Edge edge : graph.getAdjacencyList().get(u)) {
                int v = edge.destination;
                int weight = edge.weight;

                if (!visited[v] && distance[u] != Integer.MAX_VALUE &&
                    distance[u] + weight < distance[v]) {
                    distance[v] = distance[u] + weight;
                }
            }
        }

        return distance;
    }

    private static int minDistance(int[] distance, boolean[] visited) {
        int min = Integer.MAX_VALUE;
        int minIndex = -1;

        for (int v = 0; v < distance.length; v++) {
            if (!visited[v] && distance[v] <= min) {
                min = distance[v];
                minIndex = v;
            }
        }

        return minIndex;
    }
}
```

Practical Examples and Test Cases

Example Usage

```java
public class DijkstraExample {

    public static void main(String[] args) {
        // Create a graph with 6 vertices
        Graph graph = new Graph(6);

        // Add edges (source, destination, weight)
        graph.addEdge(0, 1, 4);
        graph.addEdge(0, 2, 3);
        graph.addEdge(1, 2, 1);
        graph.addEdge(1, 3, 2);
        graph.addEdge(2, 3, 4);
        graph.addEdge(3, 4, 2);
        graph.addEdge(4, 5, 6);

        int source = 0;

        // Test basic Dijkstra
        System.out.println("=== Basic Dijkstra ===");
        int[] distances = DijkstraBasic.dijkstra(graph, source);
        DijkstraBasic.printDistances(distances, source);

        // Test Dijkstra with path
        System.out.println("\n=== Dijkstra with Path ===");
        DijkstraWithPath.Result result = DijkstraWithPath.dijkstraWithPath(graph, source);

        for (int i = 0; i < graph.getVertices(); i++) {
            if (i != source) {
                List<Integer> path = DijkstraWithPath.getPath(result.previous, source, i);
                System.out.print("Distance to vertex " + i + ": " + result.distances[i] + ", ")
                DijkstraWithPath.printPath(path, source, i);
            }
        }
    }
}
```

Edge Cases and Considerations

java

```java
public class DijkstraEdgeCases {

    // Handle disconnected graphs
    public static boolean isConnected(Graph graph, int source, int destination) {
        int[] distances = DijkstraBasic.dijkstra(graph, source);
        return distances[destination] != Integer.MAX_VALUE;
    }

    // Find shortest path in a grid
    public static int shortestPathInGrid(int[][] grid, int startRow, int startCol,
                                         int endRow, int endCol) {
        int rows = grid.length;
        int cols = grid[0].length;

        // Convert 2D coordinates to 1D vertex numbers
        int vertices = rows * cols;
        Graph graph = new Graph(vertices);

        // Directions: up, down, left, right
        int[] dx = {-1, 1, 0, 0};
        int[] dy = {0, 0, -1, 1};

        // Build graph from grid
        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
                if (grid[i][j] == 0) continue; // Skip obstacles

                int currentVertex = i * cols + j;

                for (int k = 0; k < 4; k++) {
                    int newRow = i + dx[k];
                    int newCol = j + dy[k];

                    if (newRow >= 0 && newRow < rows && newCol >= 0 &&
                        newCol < cols && grid[newRow][newCol] != 0) {
                        int neighborVertex = newRow * cols + newCol;
                        graph.addEdge(currentVertex, neighborVertex, grid[newRow][newCol]);
                    }
                }
            }
        }

        int source = startRow * cols + startCol;
        int destination = endRow * cols + endCol;

        int[] distances = DijkstraBasic.dijkstra(graph, source);
```

```
        return distances[destination] == Integer.MAX_VALUE ? -1 : distances[destination];
    }
}
```

Time and Space Complexity Analysis

Complexity Breakdown

- **Time Complexity**:
  - Using Priority Queue: O((V + E) log V)
  - Using Array: $O(V^2)$
- **Space Complexity**: O(V) for distance array and priority queue
- **When to use which**:
  - Priority Queue: Better for sparse graphs ($E << V^2$)
  - Array: Better for dense graphs ($E \approx V^2$)

Performance Comparison

```java
public class PerformanceTest {

    public static void compareImplementations(Graph graph, int source) {
        long startTime, endTime;

        // Test Priority Queue implementation
        startTime = System.nanoTime();
        int[] result1 = DijkstraBasic.dijkstra(graph, source);
        endTime = System.nanoTime();
        System.out.println("Priority Queue time: " + (endTime - startTime) / 1000000.0 + " ms")

        // Test Array implementation
        startTime = System.nanoTime();
        int[] result2 = DijkstraOptimized.dijkstraArray(graph, source);
        endTime = System.nanoTime();
        System.out.println("Array-based time: " + (endTime - startTime) / 1000000.0 + " ms");

        // Verify results are the same
        System.out.println("Results match: " + Arrays.equals(result1, result2));
    }
}
```

Quiz Questions

1. What is the time complexity of Dijkstra's algorithm when implemented with a binary heap (priority queue)? a) $O(V^2)$ b) O((V + E) log V) c) O(V log V) d) O(E log E)

Correct Answer: b) O((V + E) log V)

2. Which of the following is a prerequisite for Dijkstra's algorithm to work correctly? a) The graph must be connected b) All edge weights must be non-negative c) The graph must be acyclic d) The graph must be undirected

Correct Answer: b) All edge weights must be non-negative

3. In Dijkstra's algorithm, what does the "relaxation" process do? a) Removes vertices from the graph b) Updates the shortest distance to a vertex if a shorter path is found c) Adds new edges to the graph d) Sorts the vertices by weight

Correct Answer: b) Updates the shortest distance to a vertex if a shorter path is found

4. What happens when Dijkstra's algorithm encounters a vertex that has already been visited? a) It processes the vertex again b) It skips the vertex and continues c) It throws an exception d) It restarts the algorithm

Correct Answer: b) It skips the vertex and continues

5. If we want to reconstruct the actual shortest path (not just the distance), what additional data structure do we need? a) A stack to store vertices b) A parent/previous array to track the path c) A visited array d) An additional priority queue

Correct Answer: b) A parent/previous array to track the path

Practice Coding Questions

1. Single Source Shortest Path with Path Printing Implement a method that takes a weighted directed graph, a source vertex, and a destination vertex. Return both the shortest distance and the complete path from source to destination. If no path exists, return appropriate indicators.

2. Modified Dijkstra for K Shortest Paths Write a Java program that finds the K shortest paths from a source vertex to a destination vertex in a weighted graph. The paths should be returned in ascending order of their total weights.

3. Dijkstra with Path Weight Limit Implement a variation of Dijkstra's algorithm that finds the shortest path from source to destination, but only considers paths where the total weight does not exceed a given limit. Return -1 if no such path exists.

4. Network Delay Time You are given a network of n nodes, labeled from 1 to n. You are also given a list of travel times as directed edges, where each edge is represented as (source, destination, time). Find the

minimum time it takes for a signal to reach all nodes starting from a given source node. If it's impossible to reach all nodes, return -1.

5. Shortest Path in Binary Matrix Given an n x n binary matrix grid, return the length of the shortest clear path from the top-left corner (0,0) to the bottom-right corner (n-1, n-1). A clear path is a path where all visited cells are 0. You can move in 8 directions (horizontally, vertically, and diagonally). If no clear path exists, return -1.